Exceptions në Python

Alion Emini

Kurs i Programimit në Python
kurse.informatika
September 29, 2025

Contents

1 Exceptions në Python **5**

1.1 Hyrje: çfarë është një *exception*? . . . 5

1.2 Shmangia e gabimit me kontroll paraprak . . . 6

1.2.1 Pse ky stil "shmangie" është i vlefshëm? . . . 7

1.3 Kur nuk mund të shmangen exception-et: `try/except` . . . 7

1.3.1 Si i trajtojmë: `try/except` (forma më e thjeshtë) . . . 8

1.3.2 Çfarë ndodh hap-pas-hapi në shembullin e pagës bruto? . . . 9

1.4 Files dhe Exceptions . . . 9

1.4.1 Gabim gjatë hapjes së një skedari . . . 9

1.4.2 Trajtimi i gabimit të skedarit me `try/except` . . . 9

1.5 Trajtimi i disa *exception*-eve . . . 10

1.6 Një `except` i vetëm që kap *të gjitha* exception-et . . . 11

1.7 Shfaqja e mesazhit të parazgjedhur të *exception*-it . . . 12

1.7.1 Kapje "gjithëpërfshirëse": `except Exception as err` . . . 13

1.8 Klauzola `else` . . . 14

1.9 Klauzola `finally` . . . 15

1.10 Po nëse një *exception* nuk trajtohet? . . . 16

1.11 Ngritja e një *exception*-i me `raise` . . . 16

1.12 Pasurimi i *exception*-eve me shënime (`add_note`) . . . 17

Chapter 1

Exceptions në Python

Objektivat e kapitullit

Në fund të kësaj njësie do të mund të:

- Shpjegosh çfarë është një *exception* dhe të lexosh një *traceback* (rreshti, moduli, lloji i gabimit).
- Parandalosh gabime të parashikueshme me kontrolle paraprake (p.sh. shmangia e pjesëtimit me zero).
- Trajtosh gabime me `try`/`except`, duke kapur `ValueError`, `IOError`/`FileNotFoundError` etj.
- Përdorësh shumë `except` për lloje të ndryshme dhe/ose `except Exception as e` kur kërkohet kapje e përgjithshme.
- Shfaqësh mesazhin e parazgjedhur të gabimit me `except ... as err` dhe `print(err)`.
- Aplikosh `else` (rrjedhë normale) dhe `finally` (pastrim burimesh—p.sh. mbyllje skedarësh).
- Ngresh gabime me `raise` dhe të dallosh `raise` nga `assert` (ky i fundit çaktivizohet me `-O` dhe s'duhet në prodhim).
- Pasurosh gabimet me `Exception.add_note(...)` (Python 3.11+) për kontekst diagnostikues më të mirë.

1.1 Hyrje: çfarë është një *exception*?

Një **exception** është një gabim që ndodh *gjatë ekzekutimit* të programit. Zakonisht, kur ndodh një exception, programi ndalet papritur. Më poshtë shohim një shembull të thjeshtë: programi merr dy numra nga përdoruesi dhe pjesëton të parin me të dytin. Nëse përdoruesi fut `0` si numrin e dytë, ndodh exception (pjesëtimi me zero është i pamundur matematikisht).

Programi 1 (`division1.py`)

```python
def main():
    num1 = int(input('Enter a number: '))
    num2 = int(input('Enter another number: '))

    result = num1 / num2
    print(f'{num1} divided by {num2} is {result}')

if __name__ == '__main__':
    main()
```

Output-i (1) i programit

```
Enter a number: 10
Enter another number: 2
10 divided by 2 is 5.0
```

Output-i (2) i programit me pjesëtim me zero

```
Enter a number: 10
Enter another number: 0
Traceback (most recent call last):
  File "C:\Python\division.py", line 13, in <module>
      main()
  File "C:\Python\division.py", line 9, in main
      result = num1 / num2
ZeroDivisionError: integer division or modulo by zero
```

Çfarë tregon *traceback*-u?

Traceback është mesazhi i gjatë i gabimit që shfaqet kur ngrihet (*raised*) një exception. Ai tregon:

- **Rrugën e thirrjeve** (nga moduli te funksioni ku ndodhi gabimi).
- **Numrat e rreshtave** në skedarët përkatës ku ndodhi gabimi.
- **Emrin e exception-it** (p.sh. `ZeroDivisionError`) dhe një përshkrim të shkurtër të gabimit (p.sh. `integer division or modulo by zero`).

Kur themi "u ngrit një exception", nënkuptojmë që gjatë ekzekutimit ndodhi një situatë e gabuar dhe Python e sinjalizoi atë me një objekt *exception*.

1.2 Shmangia e gabimit me kontroll paraprak

Shumë exception-e mund të parandalohen duke shkruar kod *me kujdes*. Në shembullin tonë, një `if` i thjeshtë mjafton që të mos lejojmë pjesëtim me zero. Në vend që ta lëmë exception-in të ngrihet, ne kontrollojmë vlerën e `num2` dhe shfaqim një mesazh gabimi kur është `0`. Kjo quhet shmangie *e hijshme* (graceful) e një exception-i.

Programi 2 (`division2.py`)

```python
def main():
    num1 = int(input('Enter a number: '))
    num2 = int(input('Enter another number: '))

    if num2 != 0:
        result = num1 / num2
        print(f'{num1} divided by {num2} is {result}')
    else:
        print('Cannot divide by zero.')

if __name__ == '__main__':
    main()
```

Output-i (1) i programit

```
Enter a number: 10
Enter another number: 2
10 divided by 2 is 5.0
```

Output-i (2) i programit me pjesëtim me zero

```
Enter a number: 10
Enter another number: 0
Cannot divide by zero.
```

Kujdes me pjesëtimin

Çdo tentativë për të kryer `num1 / 0` do të ngrejë `ZeroDivisionError`. Nëse rasti mund të ndodhë, parashikoje me një kontroll paraprak.

1.2.1 Pse ky stil "shmangie" është i vlefshëm?

Qasja me kontroll paraprak bën që programi të **mos përplaset** dhe përdoruesi të marrë një **mesazh të qartë**. Për shembujt didaktikë si ky, mjafton: nëse nuk mund të kryhet veprimi, i shpjegojmë përdoruesit pse. (Më vonë, kur të trajtojmë menaxhimin e gabimeve me `try/except`, do të shohim si t'i kapim këto raste edhe kur nuk mund të parashikohen lehtësisht. Këtu mjafton të *shpjegojmë më mirë* pikërisht sa na është dhënë: pjesëtimi me zero dhe leximi i *traceback*-ut.)

1.3 Kur nuk mund të shmangen exception-et: `try/except`

Edhe kur programi është shkruar me kujdes, ka raste kur *gabimet e hyrjes* (input) nuk mund t'i parandalojmë thjesht me `if`. Shembulli i mëposhtëm llogarit pagën bruto (*gross pay*) duke kërkuar orët e punës dhe pagesën orare; nëse përdoruesi fut tekst në vend të numrit, ndodh **ValueError**.

Programi 3 (`gross_pay1.py`)

```python
def main():
    hours = int(input('How many hours did you work? '))
    pay_rate = float(input('Enter your hourly pay rate: '))
    gross_pay = hours * pay_rate

    print('Gross pay: ', gross_pay)

if __name__ == '__main__':
    main()
```

```
How many hours did you work? forty
Traceback (most recent call last):
  File "C:\Users\Tony\Documents\Python\Source Code\Chapter 06\gross_pay1.py", line 17, in <
      module>
    main()
```

```
  File "C:\Users\Tony\Documents\Python\Source Code\Chapter 06\gross_pay1.py", line 5, in main
    hours = int(input('How many hours did you work? '))
ValueError: invalid literal for int() with base 10: 'forty'
```

Në këtë ekzekutim, përdoruesi shkruan `'forty'` në vend të `40`. Si pasojë, `int()` në rreshtin 5 **ngre** (*raises*) një `ValueError` dhe programi ndalet me *traceback*. Rreshti i fundit i mesazhit tregon emrin e exception-it (`ValueError`) dhe përshkrimin: `invalid literal for int() with base 10: 'forty'`.

1.3.1 Si i trajtojmë: `try/except` (forma më e thjeshtë)

Python na lejon të **kapim** exception-et dhe të reagojmë pa e rrëzuar programin, duke përdorur `try/except`. Forma bazë:

```
try:
    # try suite: një ose disa rreshta që mund të ngrejnë exception
    statement
    statement
    ...
except ExceptionName:
    # handler: ekzekutohet vetëm kur ngrihet exception-i i specifikuar
    statement
    statement
    ...
```

Ekzekutimi rrjedh kështu:

- Nëse gjatë *try suite* ngrihet një exception i llojit `ExceptionName`, programi **hidhet** te blloku *handler* dhe e ekzekuton atë; pastaj vazhdon me rreshtin pas `try/except`.
- Nëse ngrihet një exception me *lloj tjetër* (që nuk përputhet me `ExceptionName`), programi ndalet me *traceback*.
- Nëse *try suite* mbaron pa exception, `except` anashkalohet.

Programi 4 (`gross_pay2.py`) — trajtimi i `ValueError`

```
def main():
    try:
        hours = int(input('How many hours did you work? '))
        pay_rate = float(input('Enter your hourly pay rate: '))
        gross_pay = hours * pay_rate

        print('Gross pay:', gross_pay)
    except ValueError:
        print('ERROR: Hours worked and hourly pay rate must be valid numbers.')

if __name__ == '__main__':
    main()
```

```
How many hours did you work? forty
ERROR: Hours worked and hourly pay rate must be valid numbers.
```

Kur `int()` dështoi të konvertojë `'forty'`, u ngrit `ValueError` dhe ekzekutimi kaloi menjëherë te `except ValueError:`, ku u shfaq një mesazh i qartë — programi **nuk** u rrëzua dhe mund të vazhdojë me hapa të tjerë pas këtij blloku.

1.3.2 Çfarë ndodh hap-pas-hapi në shembullin e pagës bruto?

Në ekzekutimin problematik, thirrja `int(input(...))` për orët kthen një *string* jo-numerik (`'forty'`). Kjo nuk mund të konvertohet në `int`, ndaj `int()` **ngre** `ValueError`. Në atë çast, ekzekutimi **del menjëherë nga** blloku `try` dhe **kërcehet te** `except ValueError:`, ku ekzekutohet *handler*-i (mesazhi "ERROR: ..."). Pjesa e mbetur e `try`-t (leximi i pagesës orare, shumëzimi dhe `print`) anashkalohet.

1.4 Files dhe Exceptions

1.4.1 Gabim gjatë hapjes së një skedari

Programi i mëposhtëm lexon emrin e një skedari dhe shfaq përmbajtjen e tij. Funksionon vetëm nëse skedari ekziston; përndryshe ngrihet një `IOError` (gabim I/O).

Programi 5 (`display_file.py`)

```python
def main():
    filename = input('Enter a filename: ')
    infile = open(filename, 'r')
    contents = infile.read()
    print(contents)

    infile.close()

if __name__ == '__main__':
    main()
```

```
Enter a filename: bad_file.txt
Traceback (most recent call last):
  File "C:\Python\display_file.py", line 21, in <module>
    main()
  File "C:\Python\display_file.py", line 9, in main
    infile = open(filename, 'r')
IOError: [Errno 2] No such file or directory: 'bad_file.txt'
```

Gabimi ngrihet në thirrjen e `open(...)`. *Traceback*-u tregon qartë se shkaku është "`No such file or directory`".

Shënim praktik

Në Python 3 modern, ky rast zakonisht ngre `FileNotFoundError` (nënklasë e `OSError`/`IOError`). Shumë materiale përdorin ende termin `IOError`. Nëse do të jesh i sigurt, mund të kapësh `FileNotFoundError` ose `OSError`.

1.4.2 Trajtimi i gabimit të skedarit me `try`/`except`

Le ta modifikojmë programin që të mos rrëzohet kur skedari mungon, por të shfaqë një mesazh miqësor.

Programi 6 (display_file2.py)

```
def main():
    filename = input('Enter a filename: ')

    try:
        infile = open(filename, 'r')
        contents = infile.read()
        print(contents)

        infile.close()
    except IOError:
        print('An error occurred trying to read the file', filename)

if __name__ == '__main__':
    main()
```

```
Enter a filename: bad_file.txt
An error occurred trying to read the file bad_file.txt
```

Rrjedha është kjo: emri i skedarit merret nga përdoruesi; brenda `try`, `open(...)` provon ta hapë. Siqë skedari nuk ekziston, ngrihet `IOError` dhe ekzekutimi **del** nga `try`, duke **kaluar** te `except IOError:` ku ekzekutohet mesazhi informues. Rreshtat e leximit/afishimit dhe `close()` brenda `try` *anashkalohen* në rast gabimi.

1.5 Trajtimi i disa *exception*-eve

Shpesh, blloku `try` përmban kod që mund të ngrejë **më shumë se një lloj** exception-i. Në këto raste, shkruaj një `except` të ndarë për secilin lloj që dëshiron të trajtosh.

Programi 7 (sales_report1.py)

```
def main():
    total = 0.0

    try:
        infile = open('sales_data.txt', 'r')

        for line in infile:
            amount = float(line)
            total += amount

        infile.close()

        print(f'{total:,.2f}')

    except IOError:
        print('An error occurred trying to read the file.')

    except ValueError:
        print('Non-numeric data found in the file.')

    except:
```

```
        print('An error occurred.')

if __name__ == '__main__':
    main()
```

Përmbajtja e skedarit `sales_data.txt` (shembull)

```
24987.62
26978.97
32589.45
31978.47
22781.76
29871.44
```

Çfarë mund të shkojë keq këtu? `open('sales_data.txt', 'r')` mund të ngrejë `IOError` nëse skedari mungon ose nuk lexohet dot. Iterimi `for line in infile` mund të hasë një problem leximi, po ashtu `IOError`. `float(line)` ngre `ValueError` nëse rreshti nuk është numër i vlefshëm (p.sh. tekst alfabetik).
Vër re tre `except`-et:

- `except IOError:` — kap problemet e skedarit dhe shfaq një mesazh specifik.
- `except ValueError:` — kap rreshta jo-numerikë.
- `except:` — kap *çdo* exception tjetër të papritur.

Kur ndodh një exception në `try`, Python kontrollon `except`-et *nga lart poshtë*. Kur gjen një lloj që përputhet, ekzekuton *handler*-in përkatës dhe **nuk** kontrollon më poshtë. Nëse asnjë nuk përputhet, do të përdoret `except:` i përgjithshëm (nëse ekziston), përndryshe programi do të rrëzohet me *traceback*.

Këshillë

Mbaje `except:` të përgjithshëm vetëm si *mjet të fundit*. Shpesh është më mirë `except Exception as e:` dhe të log-osh `e`, ose të kapësh llojet specifike që pret realisht.

1.6 Një except i vetëm që kap *të gjitha* exception-et

Ndonjëherë dëshiron thjesht të mos lejohet rrëzimi i programit, pavarësisht llojit të gabimit—p.sh. për një prototip ose për një mesazh të unifikuar gabimi. Mund ta bësh me një `except` pa lloj specifik.

Programi 8 (`sales_report2.py`)

```
def main():
    total = 0.0

    try:
        infile = open('sales_data.txt', 'r')

        for line in infile:
            amount = float(line)
            total += amount

        infile.close()
```

```
12
13          print(f'{total:,.2f}')
14      except:
15          print('An error occurred.')
16
17  if __name__ == '__main__':
18      main()
```

Output-i i programit në rast gabimi (p.sh. skedari mungon)

```
1  Enter a filename: bad_file.txt
2  An error occurred.
```

Output-i i programit në rast suksesi (skedari ekziston)

```
1  Enter a filename: sales_data.txt
2  24987.62
3  26978.97
4  32589.45
5  31978.47
6  22781.76
7  29871.44
```

Këtu `except` *pa lloj* kap çdo exception të ndodhur në bllokun `try` dhe ekzekuton të njëjtin reagim, pavarësisht shkakut. Kjo është e thjeshtë, por kujdes të mos fshehë gabime që duhen rregulluar.

1.7 Shfaqja e mesazhit të parazgjedhur të *exception*-it

Kur ngrihet një *exception*, Python krijon në memorie një *exception object* që përmban edhe **mesazhin e parazgjedhur të gabimit**. (Është i njëjti mesazh që shfaqet në fund të *traceback*-ut kur *exception*-i mbetet i patrajtuar.) Në një `except` mund t'ia **caktojmë** këtë objekt një variable me sintaksën `except ValueError as err:`. Më pas, thjesht `print(err)` për të shfaqur mesazhin e parazgjedhur.

Programi 9 (`gross_pay3.py`)

```
1  def main():
2      try:
3          hours = int(input('How many hours did you work? '))
4          pay_rate = float(input('Enter your hourly pay rate: '))
5          gross_pay = hours * pay_rate
6
7          print('Gross pay: ', gross_pay)
8      except ValueError as err:
9          # Shfaq mesazhin e parazgjedhur të gabimit për ValueError.
10         print(err)
11
12 if __name__ == '__main__':
13     main()
```

```
How many hours did you work? forty
invalid literal for int() with base 10: 'forty'
```

Kur `ValueError` ndodh brenda `try`, shkohet te `except ValueError as err`. Variablës `err` i caktohet objekti i gabimit dhe `print(err)` shfaq **mesazhin e parazgjedhur**.

Shënim

Emri i variablës (`err`) është i lirë—mund të përdorësh çdo emër. Po ashtu, mesazhi i parazgjedhur është fiks ai që do të shihje në fund të *traceback*-ut kur gabimi *nuk* kapet.

1.7.1 Kapje "gjithëpërfshirëse": `except Exception as err`

Nëse dëshiron një `except` të vetëm që kap *çdo* gabim të mundshëm në `try`, përdor llojin bazë `Exception` dhe shfaq mesazhin përkatës.

Programi 10 (`sales_report3.py`)

```python
def main():
    total = 0.0

    try:
        infile = open('sales_data.txt', 'r')

        for line in infile:
            amount = float(line)
            total += amount

        infile.close()

        print(f'{total:,.2f}')
    except Exception as err:
        # Kap ÇDO exception dhe shfaq mesazhin e tij të parazgjedhur.
        print(err)

if __name__ == '__main__':
    main()
```

Output-i i programit në rast gabimi (p.sh. skedari mungon)

```
Enter a filename: bad_file.txt
FileNotFoundError: [Errno 2] No such file or directory: 'bad_file.txt'
```

Output-i i programit në rast suksesi (skedari ekziston)

```
Enter a filename: sales_data.txt
169187.71
```

1.8 Klauzola else

`try`/`except` mund të ketë një klauzolë opsionale `else`, e cila vjen *pas* të gjitha `except`-eve. Blloku që vjen pas `else` quhet *else suite* dhe ekzekutohet **vetëm nëse** blloku `try` përfundon pa ngritur asnjë *exception*.

```
try:
    statement
    statement
except ExceptionName:
    statement
else:
    statement
```

Programi 11 (sales_report4.py)

```
def main():
    total = 0.0

    try:
        infile = open('sales_data.txt', 'r')

        for line in infile:
            amount = float(line)
            total += amount

        infile.close()
    except Exception as err:
        print(err)
    else:
        print(f'{total:,.2f}')

if __name__ == '__main__':
    main()
```

Output-i i programit në rast gabimi (p.sh. skedari mungon)

```
Enter a filename: bad_file.txt
FileNotFoundError: [Errno 2] No such file or directory: 'bad_file.txt'
```

Output-i i programit në rast suksesi (skedari ekziston)

```
Enter a filename: sales_data.txt
169187.71
```

Në këtë program, rreshti i `print` në `else` ekzekutohet vetëm kur rreshtat brenda `try` (hapja, leximi, shumëzimi) *nuk* ngrenë asnjë gabim.

1.9 Klauzola `finally`

`try`/`except` mund të ketë edhe një klauzolë opsionale `finally`, e cila vjen *pas* të gjitha `except`-eve. Blloku *finally suite* **ekzekutohet gjithmonë** pasi të ketë mbaruar `try`-u dhe (nëse ka pasur gabim) pasi të kenë mbaruar *handler*-at e `except`—pra ekzekutohet si në rast suksesi, ashtu edhe në rast dështimi. Përdoret zakonisht për **pastrim burimesh** (mbyllje skedarësh, ç'vendosje lidhjesh, etj.).

```
try:
    statement
except ExceptionName:
    statement
finally:
    # Ky bllok ekzekutohet GJITHMONË.
    cleanup_statement
```

Shembull i shkurtër i pastrimit të burimeve:

Programi 12 (`sales_report5.py`)

```
def main():
    total = 0.0
    infile = None

    try:
        infile = open('sales_data.txt', 'r')

        for line in infile:
            amount = float(line)
            total += amount

    except Exception as err:
        print(err)
    else:
        print(f'{total:,.2f}')
    finally:
        print('Executing finally clause.')

        if infile is not None:
            infile.close()

if __name__ == '__main__':
    main()
```

Output-i i programit në rast gabimi (p.sh. skedari mungon)

```
Enter a filename: bad_file.txt
FileNotFoundError: [Errno 2] No such file or directory: 'bad_file.txt'
Executing finally clause.
```

Output-i i programit në rast suksesi (skedari ekziston)

```
Enter a filename: sales_data.txt
169187.71
Executing finally clause.
```

1.10 Po nëse një *exception* nuk trajtohet?

Nëse një *exception* **nuk** kapet nga asnjë `except`, programi **ndalet** me *traceback*. Dy mënyra tipike si një gabim mbetet i patrajtuar:

1) `try/except` nuk ka një `except` me llojin e duhur të gabimit.

2) Gabimi ngrihet *jashtë* çdo blloku `try`.

Deri tani kemi parë `ZeroDivisionError`, `IOError/FileNotFoundError` dhe `ValueError`. Ka shumë lloje të tjera. Si të dish cilat të trajtosh?

- **Dokumentimi i Python-it**: përshkruan në detaje llojet e *exception*-eve dhe kur ngrihen.
- **Eksperimentimi i qëllimshëm**: ekzekuto programin dhe provoko gabime; lexo me kujdes *traceback*-un për të mësuar emrat e *exception*-eve që duhen kapur.

1.11 Ngritja e një *exception*-i me `raise`

Ka raste kur **vetë** dëshiron ta ndalosh programin sapo ndodh një kusht i padëshiruar. Këtë e bën me fjalëkyçin `raise`, duke *ngritur* një objekt *exception* (opsionalisht me mesazh të personalizuar).

Programi 13 (`low.py`) — ngritje e drejtpërdrejtë e gabimit

```python
def main():
    number = 10

    if number > 5:
        raise Exception(f"The number should not exceed 5. ({number=})")

    print(number) # Ky rresht NUK ekzekutohet nëse ngrihet exception.

if __name__ == "__main__":
    main()
```

```
Traceback (most recent call last):
  File "./low.py", line 4, in <module>
    raise Exception(f"The number should not exceed 5. ({number=})")
Exception: The number should not exceed 5. (number=10)
```

Mesazh i personalizuar

Mesazhi u ndërtua me *f-string* dhe shprehjen vetë-dokumentuese `\{number=\}`, e cila shfaq si emrin e variablës ashtu edhe vlerën e saj (`number=10`). Kjo e bën *traceback*-un më informues gjatë diagnostikimit.

Zgjidh llojin e duhur të *exception*-it

`raise Exception(...)` funksionon, por në praktikë prefero lloje më specifike (p.sh. `ValueError`, `TypeError`, `FileNotFoundError`) që e bëjnë kapjen dhe trajtimin më të saktë.

Në këtë shembull, kemi një variabël `value` që duhet të jetë numër (`int`). Para se të vazhdojmë me operacione që kërkojnë një numër, kontrollojmë tipin e saj me `isinstance`. Nëse `value` *nuk* është `int`, ngremë një `TypeError` me një mesazh të personalizuar që tregon vlerën dhe tipin aktual të variablës. Kjo është më e saktë sesa të përdorim thjesht `Exception`, sepse lloji i exception-it tregon qartë natyrën e gabimit dhe mund të kapet lehtësisht në kodin që e përdor.

```
value = "10"

if not isinstance(value, int):
    raise TypeError(f"Expected int for {value=}, got {type(value).__name__}")
```

```
Traceback (most recent call last):
  File "./example.py", line 4, in <module>
    raise TypeError(f"Expected int for {value=}, got {type(value).__name__}")
TypeError: Expected int for value='10', got str
```

1.12 Pasurimi i *exception*-eve me shënime (`add_note`)

Zakonisht një *exception* inicializohet me një mesazh që përshkruan gabimin. Ndonjëherë, pasi e ke **kapur** gabimin, do të shtosh **kontekst shtesë** (p.sh. çfarë po punohej, në cilin rresht të skedarit, çfarë vlerë pati input-i). Për këtë shërben metoda `add_note(note)`: i shton një *shënim* (*note*) objektit të gabimit. *Traceback*-u standard i shfaq shënimet, me radhë, **pas** mesazhit të gabimit.

```
try:
    raise TypeError('bad type')
except Exception as e:
    e.add_note('Add some information')
    e.add_note('Add some more information')
raise
```

```
Traceback (most recent call last):
File "<stdin>", line 2, in <module>
raise TypeError('bad type')
TypeError: bad type
Add some information
Add some more information
```

Kërkesë versioni

`Exception.add_note(...)` është e disponueshme në Python 3.11+. Në versione më të vjetra, ky API mungon.

Programi 14 (`shembull.py`)

```python
def process_file(filename):
    try:
        with open(filename) as f:
            for lineno, line in enumerate(f, 1):
                try:
                    value = float(line)
                    # Përpunim i mëtejshëm...
                except ValueError as err:
                    err.add_note(f"Gabim në rreshtin {lineno} të skedarit '{filename}': {line.
                        strip()}")
                    raise
    except Exception as e:
        e.add_note(f"Ndodhi gabim gjatë përpunimit të skedarit '{filename}'")
        raise

if __name__ == "__main__":
    process_file("shembull.txt")
```

Përmbajtja e shembull.txt

```
12.5
abc
42.0
```

Output-i i programit në rast gabimi

```
Traceback (most recent call last):
  File "shembull.py", line 8, in process_file
    value = float(line)
ValueError: could not convert string to float: 'abc'
Gabim në rreshtin 2 të skedarit 'shembull.txt': abc
```

```
Traceback (most recent call last):
  File "shembull.py", line 17, in <module>
    process_file("shembull.txt")
  File "shembull.py", line 13, in process_file
    raise
  ...
Ndodhi gabim gjatë përpunimit të skedarit 'shembull.txt'
```

Vër re sa **i vetë-mjaftueshëm** është raportimi i gabimit: pa pasur nevojë për `print` shtesë apo logger, *traceback*-u na tregon *pikërisht* ku dhe me cilat të dhëna dështoi përpunimi. Kjo qasje është veçanërisht e dobishme në skripte CLI, punë *batch* dhe etapa ETL, ku duam që dështimet të jenë të diagnostikueshme shpejt.

Përsëritje e kapitullit

Çfarë mësove:

- **Exception** = gabim gjatë ekzekutimit. *Traceback*-u tregon rrugën e thirrjeve, rreshtin dhe llojin/mesazhin e gabimit.
- Parandalimi i gabimeve me kontrolle paraprake (p.sh. shmang `/ 0`) është zgjidhja e parë.
- `try`/`except` kap dhe trajton gabimet pa rrëzuar programin.
- Mund të kesh disa `except` për lloje të ndryshme (`ValueError`, `IOError`/`FileNotFoundError`, ...).
- `except Exception as err` kap "gjithçka", por përdore si *mjet të fundit*; `print(err)` shfaq mesazhin e parazgjedhur.
- `else` ekzekutohet vetëm kur `try` s'ngre gabim; `finally` ekzekutohet *gjithmonë* (për pastrim burimesh).
- `raise` sinjalizon gjendje të pavlefshme; zgjidh llojin *specifik* të gabimit (`ValueError`, `TypeError`, ...).
- `assert` është për *debug*; çaktivizohet me `-O`, prandaj mos e përdor për kontroll logjike në prodhim.
- (Py 3.11+) `Exception.add_note(...)` pasuron gabimin me kontekst diagnostikues që shfaqet në *traceback*.
- Me skedarë: trajto *I/O* gabimet dhe prefero `with open(...) as f:` që të mbyllet *automatikisht*.